

```
In [1]: import pandas as pd
import numpy as np
import datetime

import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler, OneHotEncoder, LabelEncoder
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, confusion_matrix, r2_score, accuracy_score
```

```
In [ ]: df= pd.read_csv('mexicoair.csv')
df.head()
```

```
In [175... df.shape
```

Out[175... (26788, 15)

```
In [176... df.describe()
```

Out[176...

	direct_radiation (W/m²)	PM25_MER	PM10_MER	SO2_MER	O3_MER	NO2_MER	CO_MER	RH_MER	TMP_MER	WDR_MER	WSP_MER	is_festiv
count	26788.000000	26788.000000	26788.000000	26788.000000	26788.000000	26788.000000	26788.000000	26788.000000	26788.000000	26788.000000	26788.000000	26788.0000
mean	198.299313	21.938476	39.138806	3.653750	25.629505	30.601047	0.505975	49.909788	18.849627	184.423941	2.10482	0.0502
std	285.059521	12.670387	20.382502	5.767726	26.409113	13.298383	0.382489	19.936643	4.285339	115.759263	1.05181	0.2184
min	0.000000	1.000000	2.000000	0.000000	0.000000	0.000000	0.000000	1.000000	6.600000	0.000000	0.10000	0.0000
25%	0.000000	13.000000	24.000000	1.000000	4.000000	21.000000	0.240000	34.000000	15.828906	84.000000	1.30000	0.0000
50%	2.000000	20.000000	36.000000	2.000000	16.000000	29.000000	0.400000	51.000000	18.400000	170.000000	1.90000	0.0000
75%	383.000000	29.000000	51.000000	4.000000	40.000000	38.000000	0.670000	67.000000	21.900000	306.000000	2.70000	0.0000
max	1008.000000	186.000000	411.000000	251.000000	156.000000	132.000000	3.650000	93.000000	31.600000	360.000000	8.10000	1.0000

```
In [177... df.isnull().sum()
```

```
Out[177...  datetime                0
          direct_radiation (W/m²)  0
          PM25_MER              0
          PM10_MER              0
          SO2_MER               0
          O3_MER                0
          NO2_MER               0
          CO_MER                0
          RH_MER                0
          TMP_MER               0
          WDR_MER               0
          WSP_MER               0
          is_festival           0
          is_weekend            0
          AirQualityIndex       0
          dtype: int64
```

```
In [178... df.duplicated().sum()
```

```
Out[178... 0
```

```
In [179... df.dtypes
```

```
Out[179...  datetime                object
          direct_radiation (W/m²)  int64
          PM25_MER              float64
          PM10_MER              float64
          SO2_MER               float64
          O3_MER                float64
          NO2_MER               float64
          CO_MER                float64
          RH_MER                float64
          TMP_MER               float64
          WDR_MER               float64
          WSP_MER               float64
          is_festival           int64
          is_weekend            int64
          AirQualityIndex       int64
          dtype: object
```

as we can see we need to predict air quality index, the o/p feature is a category variable so using labelencoder for scaling and converting this into categorical variable.

```
In [180... df['is_festival'] = df['is_festival'].astype(bool)
df['is_weekend'] = df['is_weekend'].astype(bool)
df['AirQualityIndex'] = df['AirQualityIndex'].astype('category')
```

```
In [181... df['date'] = pd.to_datetime(df['datetime']).dt.date
df['year'] = pd.to_datetime(df['datetime']).dt.year
df['day'] = pd.to_datetime(df['datetime']).dt.day
df['hour'] = pd.to_datetime(df['datetime']).dt.hour
df['month'] = pd.to_datetime(df['datetime']).dt.month

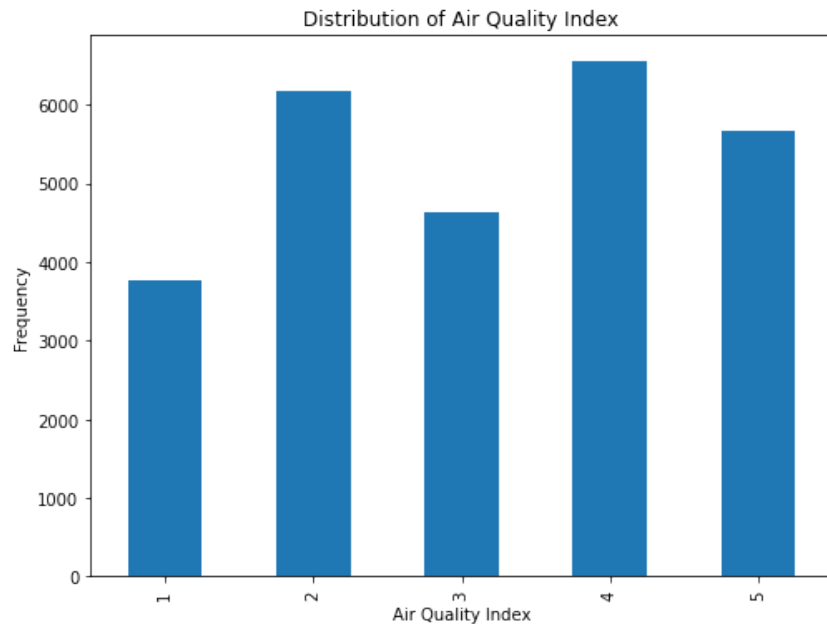
df['day_of_week'] = pd.to_datetime(df['datetime']).dt.dayofweek
```

```
In [182... df.drop(columns = 'datetime', inplace = True)
```

```
In [183... #df.to_csv('mexicoairindex.csv')
```

```
In [184... # Data analysis
```

```
In [185... plt.figure(figsize=(8, 6))
df['AirQualityIndex'].value_counts().sort_index().plot(kind='bar')
plt.title('Distribution of Air Quality Index')
plt.xlabel('Air Quality Index')
plt.ylabel('Frequency')
plt.show()
```



```
In [186... # as data is equally distributed
```

```
In [187... # Find the pollutant with the highest average value
average_pollutants = df[['PM25_MER', 'PM10_MER', 'SO2_MER', 'O3_MER', 'NO2_MER', 'CO_MER']].mean()
top_pollutant = average_pollutants.idxmax()
print(f"The most prominent pollutant is: {top_pollutant}")
```

The most prominent pollutant is: PM10_MER

```
In [188... # Analyze average pollutant levels for each Air Quality Index
aqi_factors = df.groupby('AirQualityIndex')[['PM25_MER', 'PM10_MER', 'SO2_MER', 'O3_MER', 'NO2_MER', 'CO_MER']].mean()
print("Average pollutant levels for each Air Quality Index:")
aqi_factors
```

Average pollutant levels for each Air Quality Index:

```
<ipython-input-188-b96dec763867>:2: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass ob
served=False to retain current behavior or observed=True to adopt the future default and silence this warning.
    aqi_factors = df.groupby('AirQualityIndex')[['PM25_MER', 'PM10_MER', 'SO2_MER', 'O3_MER', 'NO2_MER', 'CO_MER']].mean()
```

Out[188...

	PM25_MER	PM10_MER	SO2_MER	O3_MER	NO2_MER	CO_MER
AirQualityIndex						
1	13.811896	24.796259	2.649082	15.110753	23.090274	0.347384
2	18.997492	33.873213	4.235073	17.086154	28.970370	0.467829
3	22.336709	41.047636	4.043184	23.740685	32.321301	0.541067
4	25.458788	44.842012	3.769464	29.187413	33.385191	0.569166
5	26.132537	46.226052	3.235668	39.331551	32.731614	0.550915

PM2.5, PM10, and O₃ concentrations increase progressively from AQI 1 to AQI 5, indicating their significant contribution to worsening air quality. SO₂ shows a slight decline from AQI 3 to AQI 5, suggesting variability in its contribution to air quality levels. NO₂ concentration peaks at AQI 4 and slightly decreases at AQI 5. CO concentrations remain relatively stable but increase slightly with higher AQI levels.

In [189...

```
# Analyze thresholds for AirQualityIndex
for aqi in sorted(df['AirQualityIndex'].unique()):
    subset = df[df['AirQualityIndex'] == aqi]
    print(f"\nThresholds for Air Quality Index {aqi}:")
    print(subset[['PM25_MER', 'PM10_MER', 'SO2_MER', 'O3_MER', 'NO2_MER', 'CO_MER']].describe())
```

Thresholds for Air Quality Index 1:

	PM25_MER	PM10_MER	SO2_MER	O3_MER	NO2_MER \
count	3761.000000	3761.000000	3761.000000	3761.000000	3761.000000
mean	13.811896	24.796259	2.649082	15.110753	23.090274
std	8.436002	13.711377	4.563304	14.316639	8.651300
min	1.000000	2.000000	0.000000	0.000000	2.000000
25%	8.000000	15.000000	1.000000	4.000000	17.000000
50%	12.000000	22.000000	1.000000	11.000000	23.000000
75%	19.000000	32.000000	2.000000	21.000000	29.000000
max	69.000000	101.000000	108.000000	130.000000	72.000000

	CO_MER
count	3761.000000
mean	0.347384
std	0.252402
min	0.000000
25%	0.160000
50%	0.290000
75%	0.470000
max	3.200000

Thresholds for Air Quality Index 2:

	PM25_MER	PM10_MER	SO2_MER	O3_MER	NO2_MER \
count	6167.000000	6167.000000	6167.000000	6167.000000	6167.000000
mean	18.997492	33.873213	4.235073	17.086154	28.97037
std	10.423515	16.409485	7.905364	18.749700	10.92742
min	1.000000	2.000000	0.000000	0.000000	0.00000
25%	11.000000	22.000000	1.000000	3.000000	21.00000
50%	18.000000	32.000000	2.000000	11.000000	28.00000
75%	25.000000	44.000000	4.000000	25.000000	36.00000
max	98.000000	123.000000	251.000000	131.000000	80.00000

	CO_MER
count	6167.000000
mean	0.467829
std	0.323828
min	0.000000
25%	0.230000
50%	0.390000
75%	0.620000
max	2.700000

Thresholds for Air Quality Index 3:

	PM25_MER	PM10_MER	SO2_MER	O3_MER	NO2_MER \
count	4635.000000	4635.000000	4635.000000	4635.000000	4635.000000
mean	22.336709	41.047636	4.043184	23.740685	32.321301
std	12.173083	19.427373	5.842768	23.255180	13.311883
min	1.000000	3.000000	0.000000	0.000000	5.000000
25%	14.000000	27.000000	1.000000	4.000000	22.000000
50%	21.000000	38.447917	2.000000	17.000000	31.000000
75%	29.000000	52.000000	4.000000	37.000000	40.000000
max	146.000000	264.000000	93.000000	156.000000	97.000000

	CO_MER
count	4635.000000

mean	0.541067
std	0.410340
min	0.000000
25%	0.250000
50%	0.430000
75%	0.710000
max	3.650000

Thresholds for Air Quality Index 4:

	PM25_MER	PM10_MER	SO2_MER	O3_MER	NO2_MER \
count	6556.000000	6556.000000	6556.000000	6556.000000	6556.000000
mean	25.458788	44.842012	3.769464	29.187413	33.385191
std	13.731249	22.002571	5.243731	27.809468	14.146322
min	1.000000	2.000000	0.000000	0.000000	4.000000
25%	16.000000	29.000000	1.000000	5.000000	23.000000
50%	23.109375	42.000000	2.000000	20.000000	32.000000
75%	33.000000	57.000000	4.000000	48.000000	42.000000
max	186.000000	411.000000	87.000000	154.000000	109.000000

CO_MER	
count	6556.000000
mean	0.569166
std	0.417085
min	0.000000
25%	0.260000
50%	0.450000
75%	0.770000
max	3.610000

Thresholds for Air Quality Index 5:

	PM25_MER	PM10_MER	SO2_MER	O3_MER	NO2_MER \
count	5669.000000	5669.000000	5669.000000	5669.000000	5669.000000
mean	26.132537	46.226052	3.235668	39.331551	32.731614
std	12.992580	20.489207	3.789079	32.908866	14.955410
min	1.000000	3.000000	0.000000	0.000000	5.000000
25%	17.000000	32.000000	1.000000	7.000000	21.000000
50%	24.000000	44.000000	2.000000	35.000000	31.000000
75%	33.010417	58.000000	4.000000	65.000000	41.000000
max	114.000000	345.000000	80.000000	148.000000	132.000000

CO_MER	
count	5669.000000
mean	0.550915
std	0.412676
min	0.000000
25%	0.260000
50%	0.430000
75%	0.720000
max	2.890000

AQI 1: Represents the cleanest air quality. The mean and maximum values of pollutants are the lowest compared to other AQI levels. AQI 5: Represents the most polluted air. The mean and maximum values of pollutants are the highest in this category.

PM2.5 and PM10: These increase steadily with higher AQI levels. Higher concentrations of these fine particles indicate worsening air quality. SO2 (Sulfur Dioxide): Generally higher in AQI levels 2, 3, and 4, with a slight decrease for AQI 5. SO2 is often linked to industrial emissions and can contribute to respiratory issues. O3 (Ozone): Peaks in AQI 5, showing a significant increase as air quality worsens. High O3 levels often result from photochemical reactions and are harmful at ground level. NO2 (Nitrogen Dioxide): Gradually increases with AQI levels, indicating the contribution of vehicle emissions and industrial activities to air pollution. CO (Carbon Monoxide): Shows moderate variation but increases slightly with worsening AQI levels, indicating incomplete combustion from vehicles or other sources.

Key Insights PM2.5 and PM10:

Small particulate matter (PM2.5) has significant health impacts, including respiratory and cardiovascular problems. PM10 follows a similar trend but represents larger particles. Ozone (O3):

Levels rise significantly in AQI 4 and 5, highlighting its role in poor air quality days. CO and NO2:

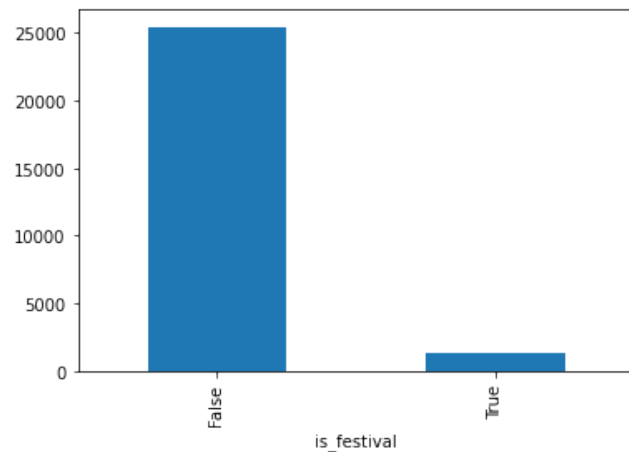
Indicators of combustion-related pollution, such as from vehicles and industrial sources. SO2:

Though its levels fluctuate, higher concentrations may correlate with industrial activity.

Higher AQI levels (4 and 5) indicate potential health risks, especially for sensitive groups.

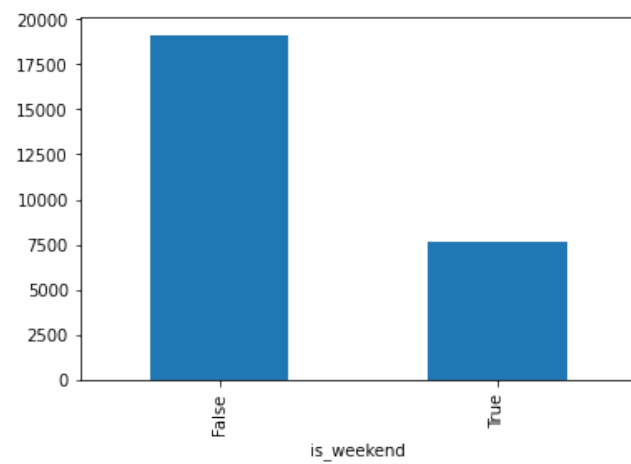
```
In [190...] df['is_festival'].value_counts().plot(kind = 'bar')
```

```
Out[190...] <Axes: xlabel='is_festival'>
```



```
In [191...] df['is_weekend'].value_counts().plot(kind= 'bar')
```

```
Out[191...] <Axes: xlabel='is_weekend'>
```



```
In [192... count = df.value_counts(df['AirQualityIndex'])  
count
```

```
Out[192... AirQualityIndex  
4    6556  
2    6167  
5    5669  
3    4635  
1    3761  
Name: count, dtype: int64
```

```
In [193... df.head
```



```
Out[193... <bound method NDFrame.head of
0      0      35.0      72.0      8.0      1.0      47.0
1      0      43.0      73.0      7.0      1.0      43.0
2      0      37.0      60.0      6.0      1.0      39.0
3      0      34.0      51.0      4.0      1.0      37.0
4      0      28.0      59.0      5.0      1.0      40.0
...      ...      ...      ...      ...      ...
26783    618      27.0      48.0      1.0     11.0      47.0
26784    423      35.0      62.0      2.0      2.0      57.0
26785    210      28.0      50.0      7.0      4.0      53.0
26786     66      43.0      59.0     12.0      6.0      47.0
26787      3      52.0      73.0      3.0      2.0      52.0
```

```
CO_MER RH_MER TMP_MER WDR_MER WSP_MER is_festival is_weekend \
0      1.80    44.0    15.6    256.0      0.5      False      False
1      1.30    47.0    15.0    296.0      0.7      False      False
2      0.90    49.0    14.4    302.0      0.5      False      False
3      0.40    51.0    13.6    320.0      0.7      False      False
4      0.70    56.0    12.6     90.0      1.2      False      False
...      ...      ...      ...      ...      ...      ...
26783    0.69    50.0    15.5    147.0      1.6      False      True
26784    1.13    53.0    14.9    183.0      1.4      False      True
26785    1.26    53.0    14.5    278.0      1.6      False      True
26786    0.89    54.0    13.8    298.0      1.6      False      True
26787    1.30    56.0    14.5    316.0      0.6      True      False
```

```
AirQualityIndex date year day hour month day_of_week
0      4  2020-11-25  2020  25   1   11           2
1      4  2020-11-25  2020  25   2   11           2
2      4  2020-11-25  2020  25   3   11           2
3      4  2020-11-25  2020  25   4   11           2
4      4  2020-11-25  2020  25   5   11           2
...      ...      ...      ...      ...      ...      ...
26783    5  2023-12-30  2023  30  21   12           5
26784    5  2023-12-30  2023  30  22   12           5
26785    5  2023-12-30  2023  30  23   12           5
26786    4  2023-12-31  2023  31   0   12           6
26787    5  2024-01-01  2024   1   0   12           0
```

```
[26788 rows x 20 columns]>
```

```
In [194... import pandas as pd

# Group by 'hour' and 'AirQualityIndex' to count occurrences
hourly_aqi = df.groupby(['hour', 'AirQualityIndex']).size().reset_index(name='count')

# Pivot the data for easier analysis
hourly_aqi_pivot = hourly_aqi.pivot(index='hour', columns='AirQualityIndex', values='count').fillna(0)

# Display the pivot table for better visualization
print(hourly_aqi_pivot)
worst_air_quality_hours = hourly_aqi_pivot[5].idxmax()
print(f'Hour with worst air quality (AirQualityIndex = 5): {worst_air_quality_hours}')
```

AirQualityIndex	1	2	3	4	5
0	247	382	198	220	73
1	281	383	182	201	69
2	290	354	166	198	108
3	295	353	152	192	124
4	253	324	158	205	176
5	231	302	143	203	237
6	229	311	142	211	223
7	209	324	160	242	181
8	175	328	196	261	156
9	151	284	225	287	169
10	130	261	222	331	172
11	115	234	225	366	176
12	92	181	201	422	220
13	81	132	137	353	413
14	69	120	97	266	564
15	74	114	79	201	648
16	104	213	217	370	212
17	98	204	163	332	319
18	73	165	147	276	455
19	84	178	225	311	318
20	86	211	308	302	209
21	101	220	319	290	186
22	127	269	299	268	153
23	166	320	274	248	108

Hour with worst air quality (AirQualityIndex = 5): 15

<ipython-input-194-6274a04891e2>:5: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and silence this warning.

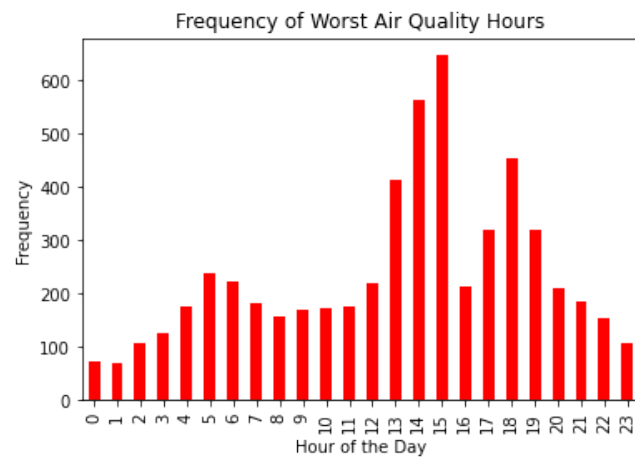
```
hourly_aqi = df.groupby(['hour', 'AirQualityIndex']).size().reset_index(name='count')
```

In [195...

```
# Filter for the highest category
high_risk = df[df['AirQualityIndex'] == 5]

# Group by hour
high_risk_hourly = high_risk['hour'].value_counts().sort_index()

# Plot
high_risk_hourly.plot(kind='bar', color='red')
plt.title('Frequency of Worst Air Quality Hours')
plt.xlabel('Hour of the Day')
plt.ylabel('Frequency')
plt.show()
```



Hour with worst air quality (AirQualityIndex = 5): 15

```
In [196... # Group by month and AirQualityIndex
monthly_aqi = df.groupby(['month', 'AirQualityIndex']).size().reset_index(name='count')
monthly_aqi_pivot = monthly_aqi.pivot(index='month', columns='AirQualityIndex', values='count').fillna(0)
print(monthly_aqi_pivot)

# Group by day of the week and AirQualityIndex
day_of_week_aqi = df.groupby(['day_of_week', 'AirQualityIndex']).size().reset_index(name='count')
day_of_week_aqi_pivot = day_of_week_aqi.pivot(index='day_of_week', columns='AirQualityIndex', values='count').fillna(0)
print(day_of_week_aqi_pivot)
```

AirQualityIndex	1	2	3	4	5
month					
1	189	310	356	714	616
2	122	379	373	517	553
3	112	458	537	684	441
4	153	410	464	616	493
5	239	492	553	563	385
6	582	617	298	424	239
7	383	643	323	445	390
8	461	621	266	495	389
9	627	654	288	360	231
10	497	514	331	422	468
11	225	540	415	533	566
12	171	529	431	783	898
day_of_week					
0	588	926	587	975	763
1	473	916	651	910	913
2	480	847	612	962	940
3	529	788	637	927	936
4	559	783	693	917	842
5	496	900	737	961	745
6	636	1007	718	904	530

```
<ipython-input-196-262adee2ff61>:2: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and silence this warning.
```

```
monthly_aqi = df.groupby(['month', 'AirQualityIndex']).size().reset_index(name='count')
```

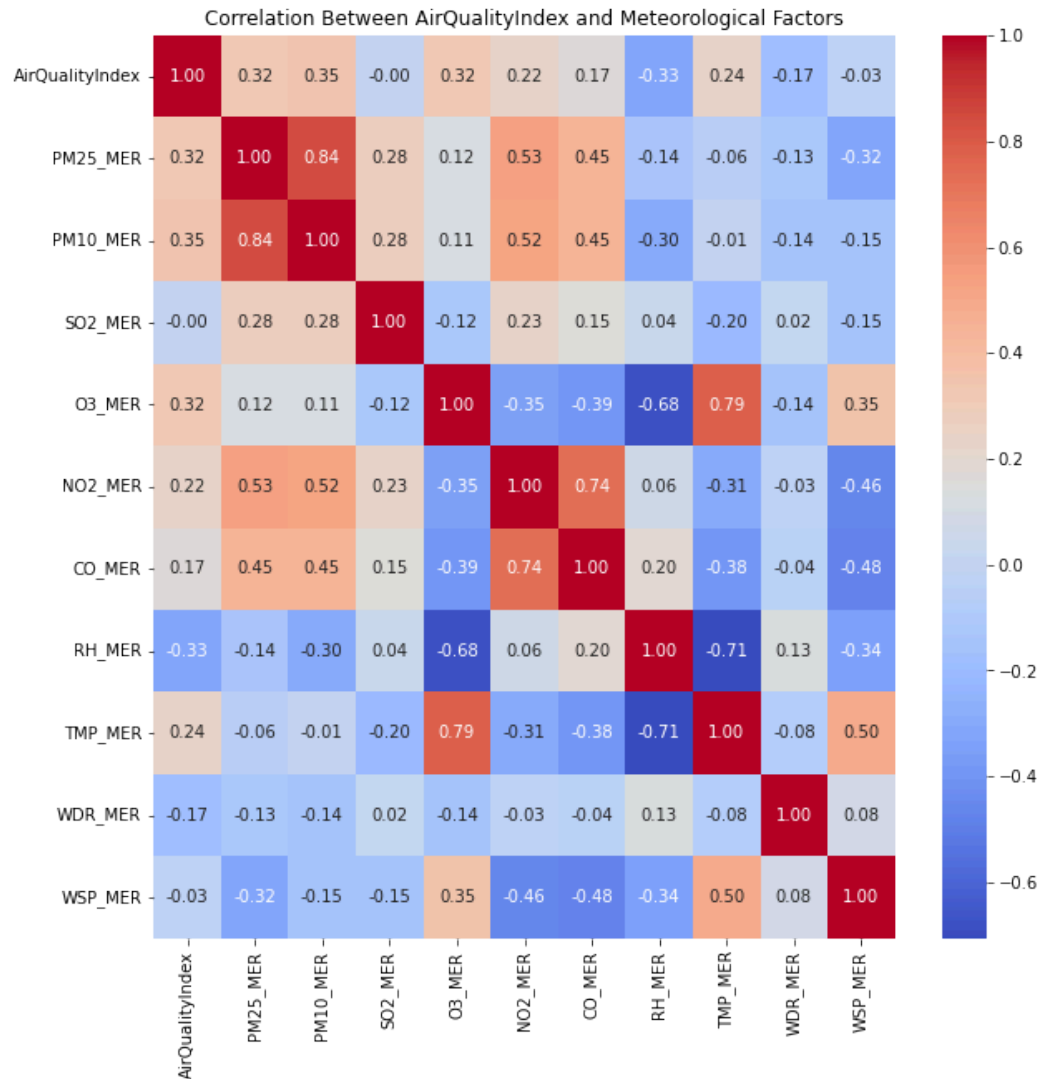
```
<ipython-input-196-262adee2ff61>:7: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and silence this warning.
```

```
day_of_week_aqi = df.groupby(['day_of_week', 'AirQualityIndex']).size().reset_index(name='count')
```

In [197...

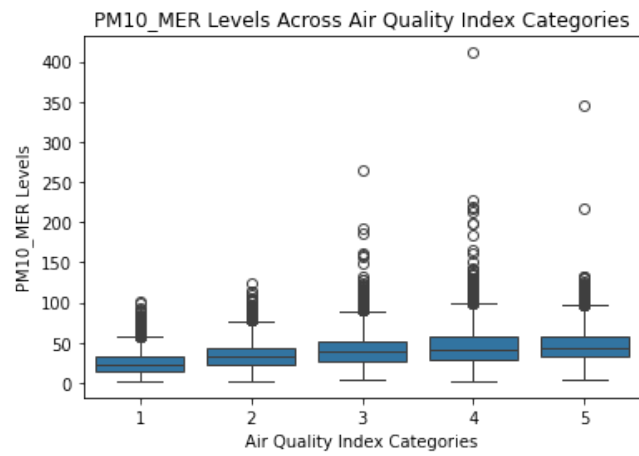
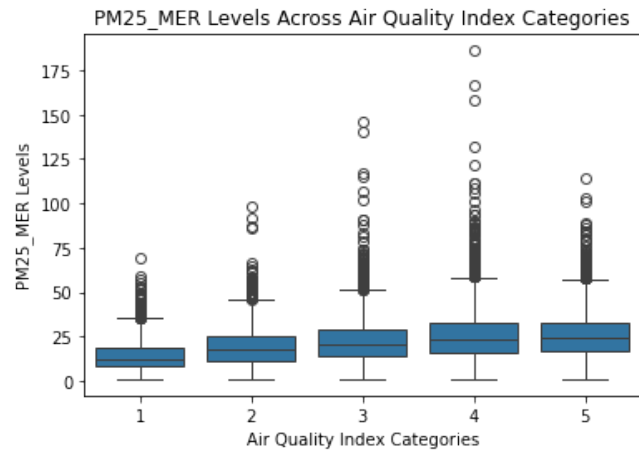
```
# Correlation matrix
correlation_matrix = df[['AirQualityIndex', 'PM25_MER', 'PM10_MER', 'SO2_MER', 'O3_MER', 'NO2_MER', 'CO_MER', 'RH_MER', 'TMP_MER', 'WDR_MER', 'WSP_MER']].corr()

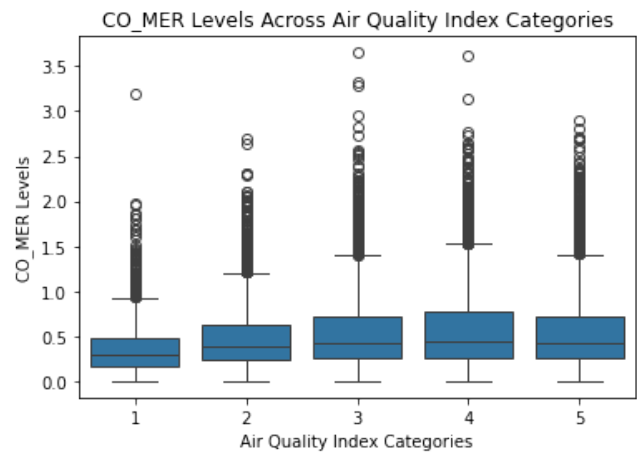
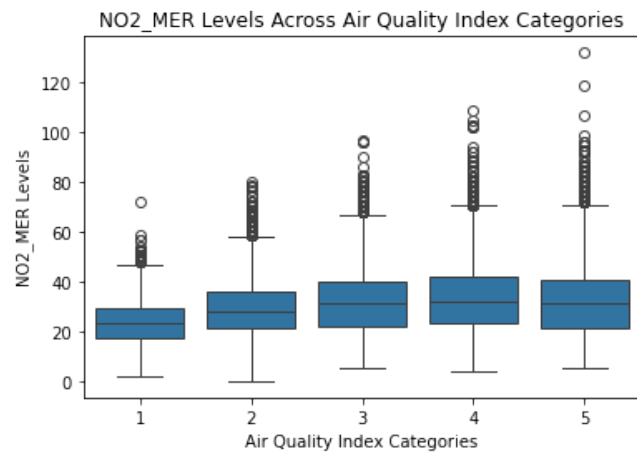
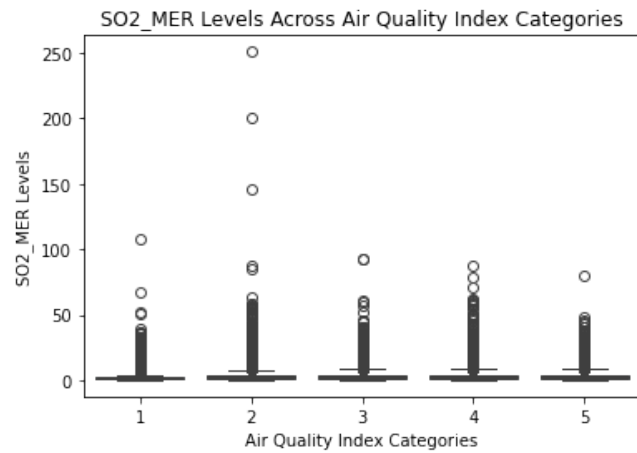
# Plot heatmap
plt.figure(figsize=(10, 10))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt='.2f')
plt.title('Correlation Between AirQualityIndex and Meteorological Factors')
plt.show()
```



In [198...

```
# Plot boxplots for each pollutant
pollutants = ['PM25_MER', 'PM10_MER', 'SO2_MER', 'NO2_MER', 'CO_MER']
for pollutant in pollutants:
    sns.boxplot(x='AirQualityIndex', y=pollutant, data=df)
    plt.title(f'{pollutant} Levels Across Air Quality Index Categories')
    plt.xlabel('Air Quality Index Categories')
    plt.ylabel(f'{pollutant} Levels')
    plt.show()
```

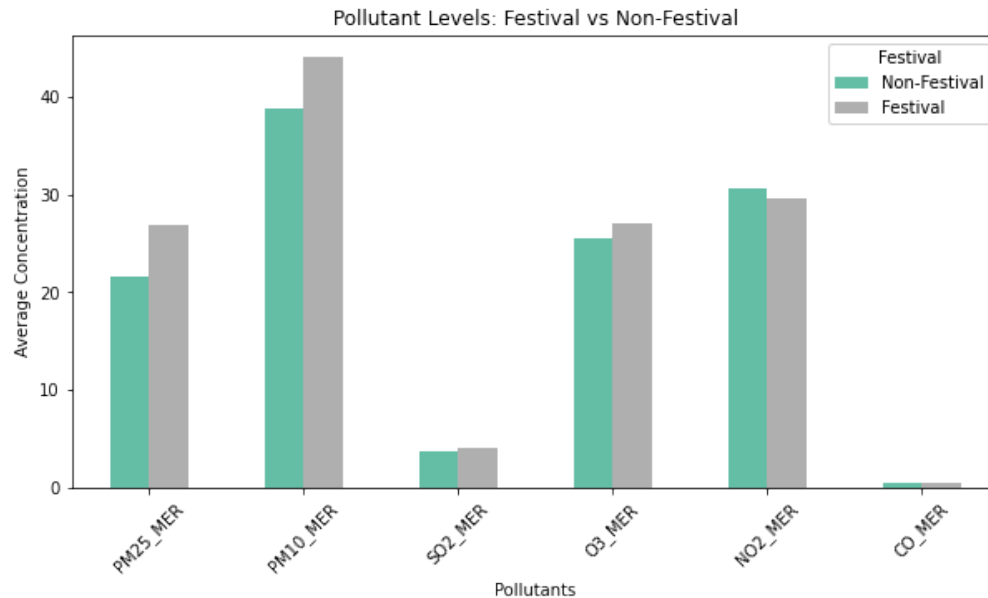




```
In [199... # Average pollutant levels during festivals vs non-festivals
festival_pollutants = df.groupby('is_festival')[['PM25_MER', 'PM10_MER', 'SO2_MER', 'O3_MER', 'NO2_MER', 'CO_MER']].mean()
print(festival_pollutants)
```

```
# Plot
festival_pollutants.T.plot(kind='bar', figsize=(10, 5), colormap='Set2')
plt.title('Pollutant Levels: Festival vs Non-Festival')
plt.ylabel('Average Concentration')
plt.xlabel('Pollutants')
plt.xticks(rotation=45)
plt.legend(['Non-Festival', 'Festival'], title='Festival')
plt.show()
```

	PM25_MER	PM10_MER	SO2_MER	O3_MER	NO2_MER	CO_MER
is_festival						
False	21.673872	38.877110	3.632372	25.556128	30.652662	0.506514
True	26.939992	44.085369	4.057841	27.016469	29.625433	0.495779



In []:

In [200... *# Interactive chart by vegalite altire*

```
In [201... import altair as alt
import pandas as pd
air_quality_count = df.groupby(['hour', 'AirQualityIndex']).size().reset_index(name='count')

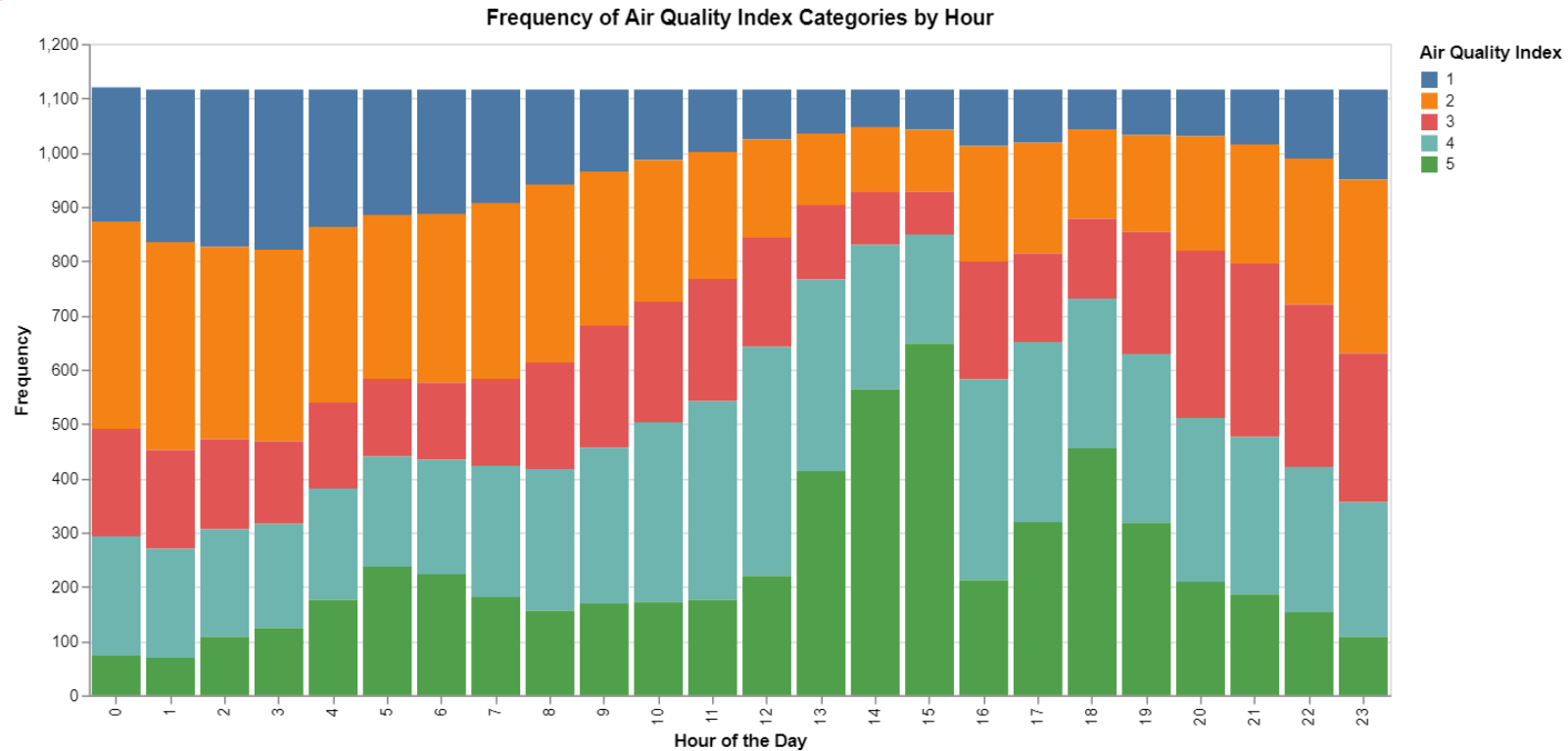
# Interactive bar chart
chart = alt.Chart(air_quality_count).mark_bar().encode(
    x=alt.X('hour:O', title='Hour of the Day'),
    y=alt.Y('count:Q', title='Frequency'),
    color=alt.Color('AirQualityIndex:N', title='Air Quality Index'),
    tooltip=['hour:O', 'AirQualityIndex:N', 'count:Q']
).properties(
    title='Frequency of Air Quality Index Categories by Hour',
    width=800,
    height=400
).interactive()
```

chart

```
<ipython-input-201-98f7c7c969da>:3: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and silence this warning.
```

```
air_quality_count = df.groupby(['hour', 'AirQualityIndex']).size().reset_index(name='count')
```

Out[201...

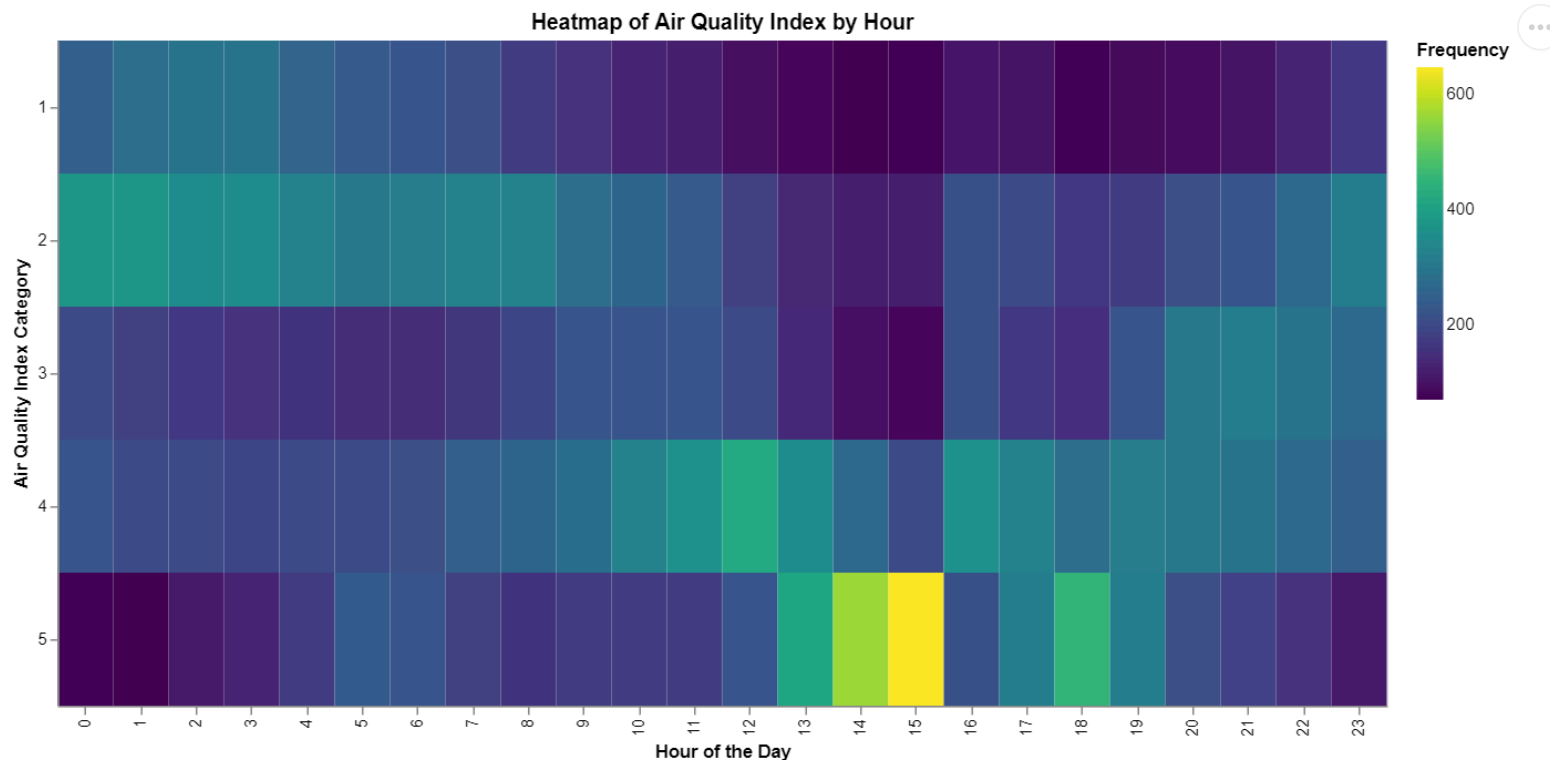


In [202...

```
# Heatmap
heatmap = alt.Chart(air_quality_count).mark_rect().encode(
    x=alt.X('hour:0', title='Hour of the Day'),
    y=alt.Y('AirQualityIndex:N', title='Air Quality Index Category'),
    color=alt.Color('count:Q', scale=alt.Scale(scheme='viridis'), title='Frequency'),
    tooltip=['hour:0', 'AirQualityIndex:N', 'count:Q']
).properties(
    title='Heatmap of Air Quality Index by Hour',
    width=800,
    height=400
).interactive()

heatmap
```


Out[202...



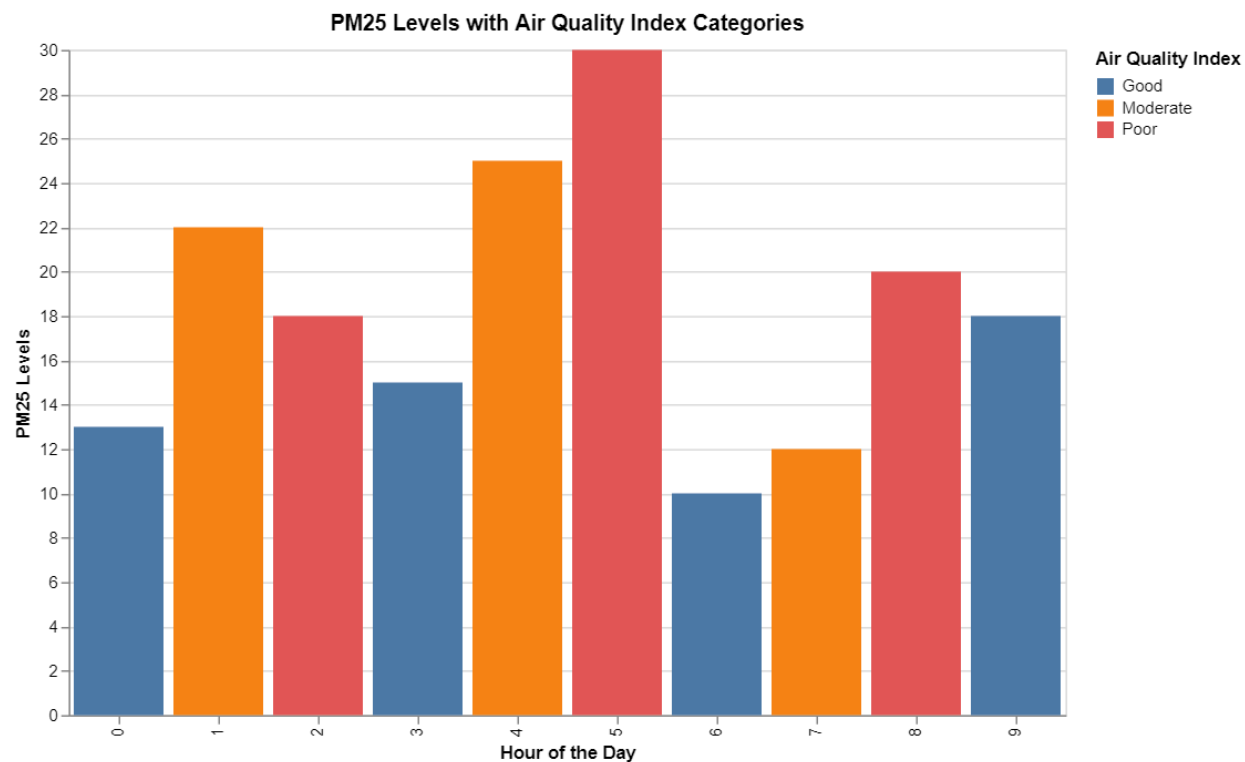
Feature Engineering

In [208...

```
categories = {1: 'Good', 2: 'Moderate', 3: 'Unhealthy for Sensitive Groups', 4: 'Unhealthy', 5: 'Very Unhealthy'}  
df['AirQualityIndex'] = df['AirQualityIndex'].map(categories)
```

In [207...

```
chart = alt.Chart(df).mark_bar().encode(  
    x=alt.X('hour:0', title='Hour of the Day'),  
    y=alt.Y('PM25_MER:Q', title='PM25 Levels'),  
    color=alt.Color('AirQualityIndex:N', title='Air Quality Index'),  
    tooltip=['hour', 'AirQualityIndex', 'PM25_MER']  
)  
.properties(  
    title="PM25 Levels with Air Quality Index Categories",  
    width=600,  
    height=400  
)  
  
# Display the chart  
chart.show()
```



```
In [25]: from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Define features (X) and target (y)
X = df[['PM25_MER', 'PM10_MER', 'SO2_MER', 'O3_MER', 'NO2_MER', 'CO_MER', 'RH_MER', 'TMP_MER', 'WDR_MER', 'WSP_MER']]
y = df['AirQualityIndex']

# Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Train a Random Forest model
model = RandomForestClassifier(random_state=42)
model.fit(X_train, y_train)

# Feature importance
importances = model.feature_importances_
feature_importance = pd.DataFrame({'Feature': X.columns, 'Importance': importances}).sort_values(by='Importance', ascending=False)

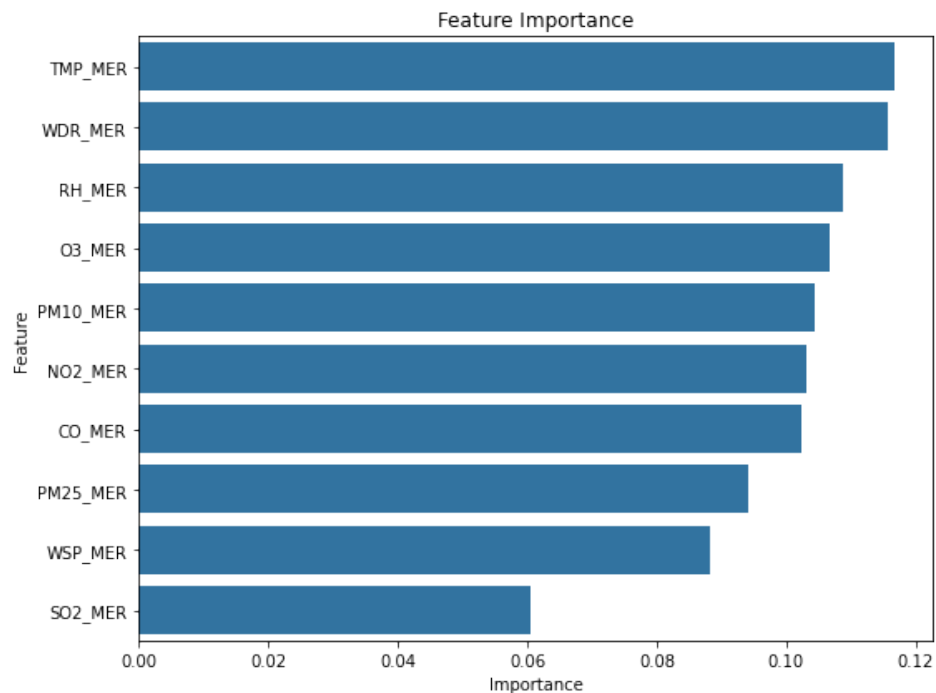
# Print feature importance
print("Feature Importance:")
print(feature_importance)

# Plot feature importance
```

```
plt.figure(figsize=(8, 6))
sns.barplot(x='Importance', y='Feature', data=feature_importance)
plt.title('Feature Importance')
plt.xlabel('Importance')
plt.ylabel('Feature')
plt.tight_layout()
plt.show()
```

Feature Importance:

	Feature	Importance
7	TMP_MER	0.116715
8	WDR_MER	0.115703
6	RH_MER	0.108736
3	O3_MER	0.106594
1	PM10_MER	0.104310
4	NO2_MER	0.103116
5	CO_MER	0.102236
0	PM25_MER	0.094105
9	WSP_MER	0.088096
2	SO2_MER	0.060390



In [26]: *# as per the algorithm all features are equally important*

The feature with the highest importance is TMP_MER (Temperature) with a score of 0.1167. This indicates that temperature has the strongest influence on determining AQI levels in your model, likely due to its effect on pollutant formation and dispersion.

In [27]: `from sklearn.inspection import permutation_importance`

```
# Train a Random Forest model
model = RandomForestClassifier(random_state=42)
model.fit(X_train, y_train)
```

```

# Permutation importance
perm_importance = permutation_importance(model, X_test, y_test, random_state=42)
feature_importance_p = pd.DataFrame({'Feature': X.columns, 'Importance': perm_importance.importances_mean}).sort_values(by='Importance', ascending=False)

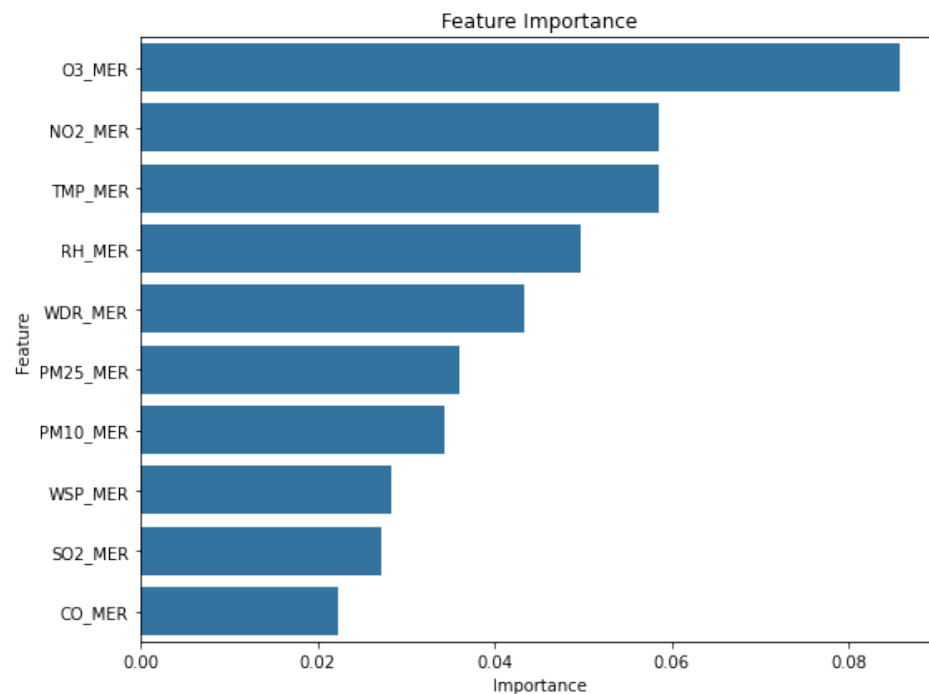
# Print feature importance
print("Permutation Feature Importance:")
print(feature_importance_p)

# Plot feature importance
plt.figure(figsize=(8, 6))
sns.barplot(x='Importance', y='Feature', data=feature_importance_p)
plt.title('Feature Importance')
plt.xlabel('Importance')
plt.ylabel('Feature')
plt.tight_layout()
plt.show()

```

Permutation Feature Importance:

	Feature	Importance
7	TMP_MER	0.116715
8	WDR_MER	0.115703
6	RH_MER	0.108736
3	O3_MER	0.106594
1	PM10_MER	0.104310
4	NO2_MER	0.103116
5	CO_MER	0.102236
0	PM25_MER	0.094105
9	WSP_MER	0.088096
2	SO2_MER	0.060390



```
In [29]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.inspection import permutation_importance
from sklearn.metrics import classification_report

# Define features (X) and target (y)
X = df[['PM25_MER', 'PM10_MER', 'SO2_MER', 'O3_MER', 'NO2_MER', 'CO_MER', 'RH_MER', 'TMP_MER', 'WDR_MER', 'WSP_MER']]
y = df['AirQualityIndex']

# Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Initialize results dictionary
feature_importance_results = {}

# 1. Random Forest
rf_model = RandomForestClassifier(random_state=42)
rf_model.fit(X_train, y_train)
rf_importances = rf_model.feature_importances_
feature_importance_results['Random Forest'] = rf_importances

# 2. Logistic Regression
logistic_model = LogisticRegression(random_state=42, max_iter=1000)
logistic_model.fit(X_train, y_train)
```

```

logistic_importances = np.abs(logistic_model.coef_).mean(axis=0)
feature_importance_results['Logistic Regression'] = logistic_importances

# 3. Permutation Importance (using Random Forest)
perm_importances = permutation_importance(rf_model, X_test, y_test, random_state=42)
perm_importances_mean = perm_importances.importances_mean
feature_importance_results['Permutation Importance'] = perm_importances_mean

# Combine feature importance results into a DataFrame
importance_df = pd.DataFrame(feature_importance_results, index=X.columns)

# Normalize importance values for better comparison
importance_df = importance_df.div(importance_df.sum(axis=0), axis=1)

# Plot feature importances
plt.figure(figsize=(10, 6))
sns.heatmap(importance_df, annot=True, cmap='coolwarm', fmt=".2f")
plt.title('Normalized Feature Importances', fontsize=16)
plt.ylabel('Features', fontsize=12)
plt.xlabel('Models', fontsize=12)
plt.show()

# Evaluate model (Random Forest) for reference
y_pred = rf_model.predict(X_test)
print("Classification Report for Random Forest:")
print(classification_report(y_test, y_pred))

```

c:\users\nishu\appdata\local\programs\python\python39\lib\site-packages\sklearn\linear_model_logistic.py:469: ConvergenceWarning: lbfgs failed to converge (status=1):

STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.

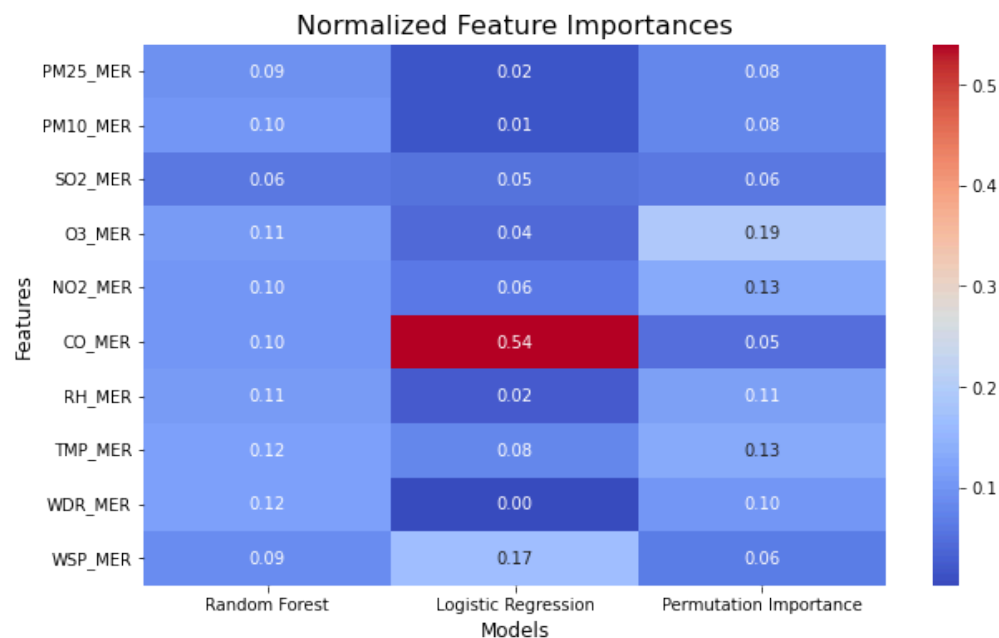
Increase the number of iterations (max_iter) or scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

n_iter_i = _check_optimize_result(



Classification Report for Random Forest:

	precision	recall	f1-score	support
Good	0.57	0.52	0.55	785
Moderate	0.38	0.51	0.44	1174
Unhealthy	0.41	0.42	0.42	1379
Unhealthy for Sensitive Groups	0.37	0.20	0.26	937
Very Unhealthy	0.48	0.51	0.49	1083
accuracy			0.44	5358
macro avg	0.44	0.43	0.43	5358
weighted avg	0.44	0.44	0.43	5358

1. Heatmap of Normalized Feature Importances This shows how important each feature (e.g., PM25_MER, PM10_MER, CO_MER, etc.) is for three methods:

Random Forest: Measures feature importance based on how much each feature contributes to reducing impurity. Logistic Regression: Indicates feature importance using the absolute value of the model's coefficients. Permutation Importance: Assesses feature importance by measuring the model's performance drop when the feature values are shuffled.

Observations:

CO_MER is most important for Logistic Regression (red at 0.54), showing its strong influence in distinguishing the target variable in a linear model. TMP_MER and WDR_MER are relatively more important in Random Forest compared to Logistic Regression. O3_MER shows the highest importance in Permutation Importance, suggesting its relevance when the model's performance is tested without that feature. Across all models, the importance of features like PM25_MER and PM10_MER is moderate.

2. Classification Report for Random Forest This provides a detailed evaluation of the Random Forest model's performance for a multi-class classification problem.

Metrics for Each Class: Precision: The proportion of correctly predicted instances for a class out of all instances predicted as that class. Example: Good has a precision of 0.57, meaning 57% of predictions labeled as Good were correct. Recall: The proportion of actual instances of a class correctly predicted. Example: Good has a recall of 0.52, meaning 52% of actual

Good instances were correctly classified. F1-Score: The harmonic mean of precision and recall. A balanced measure for model performance. Example: Good has an F1-Score of 0.55. Aggregate Metrics: Accuracy: Overall, 44% of predictions are correct. Macro Average: The unweighted average of precision, recall, and F1-Score across all classes. Example: Macro avg F1-Score is 0.43, showing average performance across classes. Weighted Average: The average of precision, recall, and F1-Score weighted by the number of samples in each class. Example: Weighted avg F1-Score is 0.44, indicating overall model performance considering class imbalance. Key Takeaways: The Random Forest model has moderate precision and recall but struggles with certain classes like Moderate and Unhealthy for Sensitive Groups, as shown by their lower F1-Scores. CO_MER is critical for linear classification models like Logistic Regression, but Random Forest gives more balanced importance to a wider range of features. Permutation Importance highlights features like O3_MER, which might be highly relevant for overall predictions.